

PROPOZYCJE PROJEKTÓW 2025/26

INFORMATYKA – STUDIA II STOPNIA

CEL PROJEKTU

Celem projektu jest wykorzystanie technik poznanych na zajęciach w większym, samodzielnym zadaniu. W szczególności chodzi o następujące techniki/algorytmy/modely:

- Preprocessing, normalizacja, augmentacja danych
- Klasyfikacja prostymi klasyfikatorami (NB, kNN, DecTree)
- Klasyfikacja sieciami neuronowymi
- Klasyfikacja obrazów konwolucyjnymi sieciami neuronowymi (i ewentualnie innymi modelami)
- Generatywne AI – grafika (GAN i inne)
- Video / Audio / Tekst
- Algorytmy analizy tekstu (bag of words, wyciąganie wydźwięku / opinii, tematu z tekstu)
- Rekurencyjne sieci neuronowe (RNN, LSTM) i inne modele generujące
- Algorytmy genetyczne
- Inteligencja roju (PSO, ACO)
- Reguły asocjacyjne
- Sterowanie agentami: Reinforcement Learning, Logika rozmyta, Gymnasium
- Transfer learning / Object Detection

Projekt można zrealizować na dwa sposoby:

RAPORT BADAWCZY

100% BADANIA AI

Dla wybranego problemu/bazy danych/zagadnienia – sprawdzam jakie modele/techniki/algorytmy działają. Próbuję osiągnąć jak najlepsze wyniki. Robię porównanie wydajności w formie sprawozdania. W tym typie projektu punkt ciężkości spada na mnogość modeli, analizę porównawczą, dochodzenie co działa, a co nie. Istotny jest dobry raport dokumentujący wszystkie wykonane eksperymenty.

APLIKACJA/SERWIS

50% BADANIA AI, 50% TWORZENIE APKI

Tworzę prototyp aplikacji/serwisu wykorzystujący AI do osiągnięcia jakiegoś celu (klasyfikacja sfotografowanego obiektu? Wyznaczenie optymalnej trasy?). W tym projekcie nadal wymagana jest pewna „naukowość” i „eksperymentacja”, ale może być mniejsza niż w przypadku raportu badawczego. Część naszego czasu będzie poświęcona na pracę nad aplikacją. Raport badawczy można zastąpić albo krótszą prezentacją, albo dokumentacją w Markdown. Mimo, że w appce zastosujemy pewnie najlepszy model jaki udało się nam stworzyć, to warto uwzględnić dojście do tego, dlaczego uważamy go za najlepszy i co po drodze testowaliśmy.

W razie wątpliwości warto z prowadzącym zajęcia doprecyzować wymagania dot. Projektu, aby podczas oceniania nie było niemiłych niespodzianek.

REALIZACJA PROJEKTU I TERMINY

Kilka zasad realizacji projektu:

- Projekt powinien dotyczyć tematyki wskazanej wyżej.
- Projekt można realizować w dowolnym języku, Python nie jest wymagany.
- Projekt realizujemy samodzielnie. W szczególnych przypadkach (np. spory stopień skomplikowania) można za zgodą prowadzącego realizować w parach.
- Projekt powinien być unikalny dla każdej osoby (zespołu) w grupie. Prowadzący założy konwersację dla każdej grupy, gdzie należy rezerwować swój temat (kto pierwszy ten lepszy).
- Prowadzący może (ale nie musi) zażyczyć sobie umieszczenia projektów w repozytoriach i udzielenie dostępu.
- Czas na wykonanie projektu: do ostatnich dwóch zajęć. Na przedostatnich zajęciach z projektów rozlicza się około połowa grupy (wyznaczona przez prowadzącego), a na ostatnich zajęciach pozostałe osoby. Podział wyznacza prowadzący zajęcia.
- Forma prezentacji: ustna z pokazaniem kodu/prezentacji/sprawozdania i ewentualnie uruchomienie programów/aplikacji (demonstracja). Może być indywidualnie prezentowane prowadzącemu zajęcia, lub prezentacja na rzutniku przed grupą. Zaleca się, by przynajmniej parę osób zaprezentowało swój projekt publicznie. Prowadzący może wymagać tego od każdego.

OCENA PROJEKTU

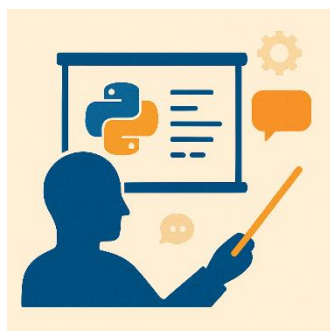
Projekt jest oceniany na 15 pkt (liczba ta może być przeskalowana) . Prowadzący wystawią ocenę w trzech aspektach:



Wyczerpujące eksperymenty i/lub dużo features w aplikacji. Czy w naszym projekcie uwzględniono badanie odpowiednio wielu modeli, czy dane były przetwarzane (preprocessing), czy przeprowadzono ewaluację?

Ocena:

- 1-2 pkt: mało modeli, algorytmów, technik
- 3-4 pkt: odpowiednia liczba modeli, algorytmów, technik, choć można było uzupełnić o dodatkowe
- 5 pkt: wyczerpująca liczba modeli, algorytmów, technik



Prezentacja, struktura, samodzielność. Czy nasz projekt opatrzony jest dobrym sprawozdaniem, lub prezentacją lub dokumentacją? Czy w trakcie prezentacji udało się odpowiedzieć na wszystkie pytania prowadzącego? Czy widać w pracy samodzielność? Czy praca została „obroniona”?

Ocena:

- 1-2 pkt: istotne braki w sprawozdaniu/prezentacji/dokumentacji, zauważalne ubytki w rozumieniu własnego kodu, mało samodzielnej pracy
- 3-4 pkt: dobra prezentacja/sprawozdanie/dokumentacja, drobne braki
- 5 pkt: prezentacja/sprawozdanie/dokumentacja na wysokim poziomie, bardzo dobra ustna prezentacja i wybronienie się z wszystkich pytań



Aspekt badawczy. Czy wykroczyliśmy poza materiał z wykładu? Czy wykazaliśmy ciekawość i inicjatywę w zgłębianiu tematyki AI?

Ocena:

- 1-2 pkt: zrealizowane tylko techniki/algorytmy/modele z wykładu w bazowej formie z bazową parametryzacją
- 3-4 pkt: zrealizowane tylko techniki/algorytmy/modele z wykładu, ale ich struktura lub parametry zostały optymalnie dobrane
- 5 pkt: wyjście poza materiał wykładu, zbadanie innych algorytmów/modeli znalezionych w internecie, widoczne przegląd literatury naukowej w bibliografii

id	x	LATE	x	x	SETE	x	s	+	DATE	s	B
3	360									12.93	54
4	350									56.92	53
13	412	56 4 7	8.33	8.33	7 8 9	8.33	12.7	3.80	3 3 8	16.88	50
6	400									35.02	39
3	400									04.72	30
18	364									24.02	18
8	385									10.25	31
6	377									12.93	73
6	77									64.23	38
4	408									8.188	12
11	36									84.84	35
4	360									12.94	56
5	347									23.51	55
3	345									22.11	57
3	345									35.85	38
10	304									32.79	73
5	406									35.80	11
15	417									56.91	57
15	346									80.86	38
15	375	56 85	53.1	3.03	625 85	3.22	315	0.31	5.3 0.3	82.88	36
35	LAP	42	83	DPE	77	83	64	LPE	75		

Projekt należy zacząć od wybrania bazy danych. Jest tutaj kilka możliwości. Możemy wybrać dataset ze strony np.

- Kilka dość oklepanych przykładów z Kaggle (były jako projekty w zeszłych latach):

- Rozpoznawanie ile zarabia osoba.

- <https://www.kaggle.com/datasets/blastchar/telco-customer-churn> Klasyfikacja klientów telefonii komórkowej: czy przedłużą umowę, czy zrezygnują?
- <https://www.kaggle.com/datasets/uciml/red-wine-quality-cortez-et-al-2009> Jaką jakoś ma czerwone wino? (można zmienić liczbę klas z 1-10 na good/bad).
- <https://www.kaggle.com/datasets/johnsmith88/heart-disease-dataset> Diagnoza choroby serca.
- <https://www.kaggle.com/datasets/emmanuelwerr/thyroid-disease-data> Diagnoza choroby tarczycy.
- <https://www.kaggle.com/datasets/uciml/mushroom-classification> Klasyfikacja grzybów jadalnych i niejadalnych.

Należy taką bazę danych przebadac pod katem klasyfikacji i wyniki przedstawic w czytelnej formie w formie raportu lub prezentacji. Mozna podzielic raport na rozdzialy:

- **Baza danych:** struktura, znaczenie kolumn, co jest klasyfikowane, jakie są wartości, wyświetlenie statystyk dla kolumn lub nawet wykresów. Objaśnienie.
- **Preprocessing:** czy baza danych wymaga naprawy? Jakie są błędy? Czy są wartości odstające, brakujące? Jaki rodzaj normalizacji lub modyfikacji danych działać będzie najlepiej? Czy zbiór trzeba balansować dla klas? Jeśli tak, to jaką metodą?
- **Klasyfikacja:** testowanie różnych klasyfikatorów poznanych na zajęciach. Dla każdego klasyfikatora można rozpatrzyć różnorakie konfiguracje parametrów. Ewaluacja klasyfikatorów powinna być przeprowadzona za pomocą różnych sensownych miar. Można skorzystać z modeli pre-trained.
- **Reguły asocjacyjne:** można poszukać w bazie danych jakichś ciekawych zależności.
- **Podsumowanie i interpretacja wyników:** co wyszło, co nie wyszło? Co działa, a co nie? Czy są jakieś interesujące wnioski z badań?

Im ciekawszy, bardziej szczegółowy i dociekliwy raport, tym lepsza ocena za niego. Będą brane takie aspekty jak:

- Czy baza danych jest interesująca, oryginalna, stworzona własnoręcznie?
- Czy na bazie danych dokonano jakiegoś istotnego preprocessingu?
- Czy klasyfikatory zostały dostatecznie szczegółowo przebadane (pod kątem parametrów, miar, wersji bazy danych, krzywych uczenia, itp.)?
- Czy dodano reguły asociacyjne?

2) KLASYFIKACJA OBRAZÓW



W zasadzie opis będzie podobny do tego powyżej. Bazę danych można próbować tworzyć ręcznie lub pobrać z Kaggle. Kilka przykładów:

- <https://www.kaggle.com/datasets/gpiosenka/100-bird-species> Klasyfikacja gatunków ptaków
- <https://www.kaggle.com/datasets/phucthaiv02/butterfly-image-classification> Klasyfikacja gatunków motyli
- <https://www.kaggle.com/datasets/alexattia/the-simpsons-characters-dataset> <https://www.kaggle.com/datasets/kostastokis/simpsons-faces> Klasyfikacja postaci z Simpsonów
- <https://www.kaggle.com/datasets/andyczhao/covidx-cxr2> Diagnoza COVID na podstawie zdjęcia RTG płuc
- <https://www.kaggle.com/datasets/csafrut2/plant-leaves-for-image-classification> Klasyfikacja roślin po obrazie liścia

Istnieje wiele innych: <https://www.kaggle.com/search?q=image+classification+in%3Adatasets> (polecam przejrzeć)

Wymagania jak w propozycji A, choć preprocessing może wyglądać trochę inaczej (augmentacja zdjęć, kompresja, konwersja do szarości lub nie, filtry, itp.).

Najlepszym klasyfikatorem będzie tutaj pewnie CNN z różnymi konfiguracjami. Ale warto też potestować inne klasyfikatory (zwykłe NN, kNN) i może różne modele transfer learning.

Krzywe uczenia są tutaj mile widziane.

W ramach dodatkowego bonusu do tego projektu, można spróbować wytrenować sieć GAN do generowania obrazów z badanej bazy danych.

3) TWORZENIE OBRAZÓW GENERATYWNYM AI



Celem projektu jest przetestowanie różnych technik tworzenia grafiki za pomocą generatywnej AI. Na zajęciach poznaliśmy GAN. Chcemy rozszerzyć te badania o modele wykraczające poza materiał zajęć.

Szczegółowy opis

Celem projektu jest zapoznanie się z różnymi metodami generowania grafiki za pomocą modeli deep-learning. Studenci mają za zadanie wytrenować model generujący grafiki na podstawie wybranej bazy danych obrazów (np. z Kaggle) i porównać różne podejścia pod kątem jakości generowanych wyników, stabilności treningu i efektywności.

Zakres projektu:

1. Pobranie i przygotowanie danych:

- Studenci wybierają bazę danych grafik dostępnych na Kaggle (np. zdjęcia, obrazy rysunkowe, ikonki itp.).
- Dokonują wstępnej analizy danych oraz ich przetwarzania (normalizacja, skalowanie, augmentacja danych).

2. Implementacja modelu bazowego:

- Jako podstawowy model sugeruje się implementację **Generative Adversarial Network (GAN)**.
- Studenci muszą zaimplementować dwa komponenty GAN: generator i dyskryminator, a następnie przetestować jego działanie.

3. Testowanie alternatywnych metod:

Studenci mają możliwość zaimplementowania innych modeli generatywnych, takich jak:

- **Variational Autoencoders (VAE):** Model probabilistyczny uczący się rozkładu danych. Jest łatwiejszy w implementacji niż GAN i stabilniejszy w treningu.
- **Conditional GAN (cGAN):** Rozszerzenie GAN, które pozwala na generowanie obrazów warunkowanych na dodatkowych etykietach (np. generowanie obrazów kotów, psów lub innych obiektów w zależności od klasy).
- **Deep Convolutional GAN (DCGAN):** Usprawniona wersja GAN wykorzystująca splotowe warstwy neuronowe (Convolutional Neural Networks, CNN), co poprawia jakość generowanych obrazów.
- **StyleGAN:** Zaawansowany model do generowania realistycznych obrazów z możliwością manipulacji stylami.
- **Denoising Diffusion Probabilistic Models (DDPM):** Nowoczesny model generatywny oparty na procesie dyfuzji, pozwalający uzyskać wysoką jakość wyników.
- **Neural Radiance Fields (NeRF):** Model do generowania grafiki 3D, który może być opcjonalnie zaimplementowany w bardziej zaawansowanej wersji projektu.

4. Ewaluacja wyników:

- Jakość generowanych obrazów może być oceniana za pomocą metryk takich jak:
 - **Frechet Inception Distance (FID):** mierzący podobieństwo dystrybucji wygenerowanych i prawdziwych obrazów.
 - **Inception Score (IS):** oceniający różnorodność i jakość wygenerowanych obrazów.
- Można również przeprowadzić subiektywną ocenę jakości generowanych grafik przez zespół.

5. Raport końcowy:

- Opis wykorzystanych modeli.
- Porównanie wyników różnych metod (jeśli studenci testowali więcej niż jedną).
- Wnioski dotyczące jakości, stabilności i trudności implementacji różnych podejść.

Przydatne linki

1. **Generative Adversarial Networks (GAN):**
 - Artykuł przeglądowy: <https://arxiv.org/abs/2111.13282>
 - Kurs na Hugging Face: <https://huggingface.co/learn/computer-vision-course/en/unit5/generative-models/gans>
2. **Variational Autoencoders (VAE):**
 - Wykład "Variational Autoencoders": <https://www.cs.columbia.edu/~zemel/Class/Nndl-2021/files/lec13.pdf>
3. **Porównanie VAE i GAN:**
 - Artykuł przeglądowy: <https://arxiv.org/abs/2103.04922>
4. **StyleGAN:**
 - Artykuł na temat StyleGAN: <https://en.wikipedia.org/wiki/StyleGAN>
5. **Denoising Diffusion Probabilistic Models (DDPM):**
 - Artykuł przeglądowy: https://en.wikipedia.org/wiki/Diffusion_model
6. **Neural Radiance Fields (NeRF):**
 - Artykuł przeglądowy: https://link.springer.com/chapter/10.1007/978-981-97-2550-2_23

4) WYKRYWANIE OBIEKTÓW NA VIDEO I STEROWANIE



Fajnie by było grać w grę nie klikając w myszkę, komórkę, klawiaturę czy konsolę, a wyginając ciało śmiało 😊

Super film na zajawkę o co chodzi:

https://www.youtube.com/watch?v=Vi3Li3TkUVY&ab_channel=EverythingIsHacked

Celem projektu byłoby stworzenie appki, a właściwie jej prototypu – wystarczy zgrubnie napisany program, która będzie jakąś grą (ściągniętą lub napisaną własnoręcznie) z możliwością sterowania obiektem w grze za pomocą ruchu.

O jakich ruchach mowa? Mogą to być gest dłonią (trochę łatwiejsze), machanie jakimś przedmiotem ze znacznikami, poruszanie całym ciałem. Można dla ułatwienia

złożyć, że osoba jest w odpowiedniej odległości lub odpowiednio ubrana. Projekt jest ciekawy, ale trzeba nałożyć sobie parę zdroworozsądkowych ułatwień.

Można przetestować kilka opcji (tak jak ten gość z filmiku) – jeśli nie uda się z machaniem rękami, to może skupmy się na dłoniach? Jeśli nasz własny model się nie wytrenował to może zastosujemy transfer learning? Kilka linków do przejrzenia:

- <https://huggingface.co/datasets/sayakpaul/poses-controlnet-dataset>
- <https://huggingface.co/spaces/hysts/mediapipe-pose-estimation>
- <https://developers.google.com/mediapipe> + <https://github.com/google/mediapipe>

5) ASYNTENT ĆWICZEŃ Z KAMERĄ



1) Wprowadzenie, motywacja, inspiracje

Asystent ćwiczeń z kamerą to projekt, który próbuje zrobić “mini-trenera” na żywo: rozpoznaje ćwiczenie, liczy powtórzenia i (najciekawsze) daje prosty feedback o technice. To wdzięczny temat, bo:

- jest **widowiskowy na prezentacji** (kamera → overlay szkieletu → licznik → komunikaty),
- ma naturalny **aspekt badawczy**: można porównać kilka podejść (reguły vs model uczony, różne metryki, wpływ wygładzenia, różne ćwiczenia), co jest mocno w duchu “eksperymenty + aplikacja” z Twojego PDF-a.

Inspiracje / zajawka (YouTube):

- Tutorial + projekt liczenia powtórzeń w czasie rzeczywistym (Python, MediaPipe, OpenCV): [YouTube](#)
- “AI-Based Workout Assistant using MediaPipe and Python” (podobny vibe): [YouTube](#)
- Klasyczny “AI push-up counter” krok po kroku: [YouTube](#)

Proponowany zakres ćwiczeń (żeby było realistycznie):

- Minimum: 1–2 ćwiczenia, np. **przysiad** i **pompka** (łatwo zdefiniować “górze/dół” ruchem stawów).
- Wersja ambitna: dołóż **biceps curl** albo **shoulder press** (często są w datasetach i tutorialach). [arXiv](#)

2) Technika: biblioteki, technologie, sprzęt, dane, trenowanie

Pipeline (jak to działa “od kamery do feedbacku”)

1. **Kamera / wideo** (webcam lub nagranie) → klatki.
2. **Estymacja pozy**: wyciągasz punkty ciała (np. bark, łokieć, biodro, kolano). Najprościej: **MediaPipe Pose**. [YouTube+1](#)
3. **Wygładzenie** (żeby punkty nie “skakały”) i proste przeliczenia: kąty w stawach, odległości, proporcje.
4. **Logika aplikacji**:
 - rozpoznanie ćwiczenia (co użytkownik robi),
 - licznik powtórzeń,
 - ocena techniki i komunikat (np. “kolana uciekają do środka” / “za płytko”).

Biblioteki / narzędzia (Python)

- **MediaPipe** – punkty ciała (pose), ewentualnie dłonie. [YouTube](#)
- **OpenCV** – kamera, rysowanie overlay, zapis wideo. [YouTube](#)
- **NumPy** – obliczenia kątów, filtracja sygnałów.
- **scikit-learn** – jeśli wybierzesz klasyfikator “klasyczny” (SVM / RandomForest) na cechach z punktów.
- **PyTorch** albo **TensorFlow/Keras** – jeśli chcesz model, który rozumie ruch w czasie (np. LSTM/1D-CNN).
- Prosty interfejs:
 - **Streamlit** lub **Gradio** (demo w przeglądarce),
 - albo **Tkinter/PyQt** (desktop).

Sprzęt

- Wystarczy **zwykła kamera** w laptopie, ale pomaga:
 - statyw / stabilne ustawienie,
 - dobre światło,
 - kadr “od pasa w dół” dla przysiadów albo “cała sylwetka” dla ćwiczeń mieszanych.

Dane: własne czy gotowe?

Masz dwie sensowne ścieżki (możesz je nawet połączyć):

A) Dane własne (najbardziej “projektowe” i pod kontrolą)

- Nagrywasz krótkie klipy (np. po 20–40 s) dla 2–3 ćwiczeń.
- Zapisujesz:
 - surowe wideo,
 - oraz “tabelkę” z punktami ciała (33 landmarky MediaPipe) na każdą klatkę.
- Plus: dokładnie wiesz, co jest “poprawne” i “błędne”, możesz celowo nagrać typowe błędy.

B) Datasets publiczne (żeby szybciej ruszyć + porównać wyniki)

Przykłady datasetów, które pasują idealnie, bo często już mają ćwiczenia albo nawet landmarky:

- Kaggle: **gym exercise – MediaPipe 33 landmarks** (gotowe punkty ciała dla kilku klas). [Kaggle](#)
- Kaggle: **Fitness Pose Classification** (pozy/ćwiczenia typu push-up, pull-up, squat itd.). [Kaggle+1](#)
- Kaggle: **Squat exercise pose dataset** (nastawione na “poprawny vs błędny” przysiad). [Kaggle](#)
- Kaggle: **gym-data yolo pose** (materiały pod podejście z pozą/klatkami). [Kaggle](#)
- Praca/paper o korekcie techniki z OpenCV + MediaPipe (może posłużyć jako punkt odniesienia w raporcie). [Ki-IT](#)
- Dla ambitnych: naukowe datasety do feedbacku treningowego (np. AIFit). [Mihai Fieraru](#)

Jak trenować (prosto, bez przekombinowania)

Wersja 1: bez treningu (reguły + progі)

- Licznik powtórzeń robisz jako “stan”: góra → dół → góra, mierząc np. kąt kolana (przysiad) albo łokcia (biceps).
- Ocena techniki: kilka reguł (np. “kolano przeszło linię palców”, “plecy zaokrąglone” – ostrożnie, bo kamera 2D).

Wersja 2: trening lekki (klasyfikator na cechach)

- Z landmarków liczysz cechy (kąty, odległości, tempo) w krótkim okienku czasowym (np. 1 sekunda).
- Uczysz model: “przysiad / pompka / nic” + ewentualnie “poprawnie / błędnie”.
- To świetnie pasuje do raportu porównawczego: różne zestawy cech, różne klasyfikatory, porównanie skuteczności.

Wersja 3: model czasowy (ambitna)

- Model dostaje sekwencję landmarków (np. 60 klatek) i sam uczy się ruchu.
- Dobre do wykrywania powtórzeń i anomalii, ale wymaga więcej danych i rozsądnego treningu.

3) Końcowy produkt: minimalny zestaw + rozszerzenia

Minimum (MVP) – żeby dało się obronić i pokazać

1. **Podgląd z kamery** + nałożony szkielet (landmarky).
2. **Wybór ćwiczenia** (np. przysiad / pompka).
3. **Liczenie powtórzeń** + tempo (powtórzenia/min).
4. **1–2 komunikaty o technice** (proste, czytelne).
5. **Krótki raport po serii** (czas, liczba powtórzeń, “średnia jakość”).

Dodatki (robią efekt “wow” i dają pole do badań)

- **Tryb “ucz się mojego stylu”**: 10 poprawnych powtórzeń jako kalibracja, potem feedback bardziej dopasowany.
- **Wykrywanie typowych błędów** (dla przysiadu: koślawienie kolana, zbyt mała głębokość; dla pompki: biodra za wysoko/za nisko).
- **Porównanie 2 podejść** w raporcie: reguły vs model uczony (plus metryki i wykresy).
- **Analiza serii w czasie**: wykres jakości w kolejnych powtórzeniach (czy zmęczenie psuje technikę?).
- **Eksport**: zapis treningu do pliku (CSV/JSON) + zapis krótkich “flagowanych” fragmentów wideo.

Jeśli chcesz, mogę dopisać do tego opisu gotowy szkic “co dokładnie badać w raporcie” (jakie metryki, jakie porównania modeli, jakie wykresy), żeby idealnie podbić część “eksperymenty + wnioski” z kryteriów oceniania.



Celem projektu jest wykorzystanie LLM (large language model), do generowania odpowiedzi w specyficznym kontekście/wirtualnym środowisku. Przykłady:

- Chcemy czatbota na stronę Inf.ug.edu.pl i chatbot powinien znać wszystkie sylabusy i odpowiadać studentom lub kandydatom na studia na pytania na temat studiów.

Użytkownik: „Co przerabiane jest na przedmiocie Inteligencja obliczeniowa?”

Chatbot (LLM): „Zgodnie z sylabusem dla kierunku Informatyka, na Inteligencji obliczeniowej przerabiane jest uczenie maszynowe, algorytmy metaheurystyczne, deep learning”

Użytkownik: „Możesz wyjaśnić co to za pojęcia”

Chatbot (LLM): „Tak, oto wyjaśnienie: ...”

- Chcemy postać w grze (NPC, non-playable character) typu RPG, która będzie z nami miała interakcję lepszą niż tylko prosty dialog typu „wybierz odpowiedź a lub b”. Chcemy, żeby postać z nami rozmawiała swobodnie, ale żeby dialog pasował do świata przykład:

Gracz: „Kowalu, czy gdzie w tym mieście jest karczma?”

NPC (LLM): „Musisz iść na północ, mości panie. Karczma jest za młynem.”

Gracz: „Dzięki, ziomal. Lubisz grać na kompie?”

NPC (LLM): „Cóż mówisz, wędrowcze. Nie rozumiem tych przedziwnych słów!”

Kolejna rozmowa można nawiązywać do poprzedniej, o ile jest zapamiętana:

NPC (LLM): „O znowu przybywasz. Mów tym razem bardziej zrozumiale, mości panie”

Projekt można zrobić naprawdę ambitnie, testując różne metody i robiąc wszystko „od podszewki”. Można też skorzystać z gotowych skryptów i rozwiązań, co będzie sporym ułatwieniem. Jeśli ktoś podejrze do tego ambitniej, możemy się umówić, że ten projekt zaliczy oba projekty. W przypadku sporego zaangażowania, można pewnie to rozwinąć w kierunku pracy mgr lub napisać raport naukowy (w czym mogę pomóc i się trochę zaangażować).

Przykłady Projektów i Technik z linkami (podpowiedzi od ChatGPT 😊, warto dokładniej przeszukać internet).

1. Dynamiczna generacja dialogów

Modele LLM, takie jak GPT-3 i GPT-4, mogą tworzyć dialogi kontekstowe zamiast korzystać z sztywnych linii dialogowych. Dzięki temu NPC są bardziej naturalni i immersyjni.

https://lutpub.lut.fi/bitstream/handle/10024/167809/bachelorthesis_Huang_Junyang.pdf?sequence=1

2. Dialogi świadome kontekstu

NPC generują odpowiedzi uwzględniające stan gry lub wcześniejsze interakcje z graczem, co zwiększa zaangażowanie.

https://projekter.aau.dk/projekter/files/536738243/The_Effect_of_Context_aware_LLM_based_NPC_Dialogues_on_Player_Engagement_in_Role_playing_Video_Games.pdf

3. Role-playing w modelach językowych

NPC są projektowani z określonymi „rolami”, co pozwala na realistyczne zachowania i spójność w interakcjach.

<https://arxiv.org/abs/2305.16367>

4. Interaktywne rozmowy z NPC w grach

Wytyczne dla projektantów pokazują, jak wykorzystać LLM, by NPC generowali odpowiedzi adekwatne do sytuacji w grze.

https://link.springer.com/chapter/10.1007/978-3-031-54975-5_10

5. Generowanie narracji proceduralnych

Framework PANGeA używa LLM do tworzenia narracji i postaci NPC z cechami osobowości, np. zgodnie z modelem Wielkiej Piątki.

<https://arxiv.org/abs/2404.19721>

6. AI Dungeon i mody AI w grach

Gry jak „AI Dungeon” oraz mody do „Skyrim” i „Stardew Valley” pokazują, jak LLM mogą tworzyć nieograniczone rozmowy z graczami.

- AI Dungeon: https://en.wikipedia.org/wiki/AI_Dungeon

- Mody w grach: <https://www.theverge.com/2024/10/17/24268007/modders-ai-companions-stardew-valley-skyrim>

7. Hugging Face – Fine-Tuning LLM dla Twojego Chatbota

Hugging Face oferuje szczegółowy przewodnik, jak dostroić istniejący model językowy (np. GPT-2, GPT-3) na własnym zestawie danych, co pozwala stworzyć spersonalizowanego chatbota.

Link: <https://huggingface.co/blog/how-to-train>

8. **OpenAI – Tworzenie zaawansowanych promptów z API GPT**
OpenAI dokumentuje, jak korzystać z prompt-engineering, by sterować zachowaniem modeli GPT bez konieczności fine-tuning. Zawiera przykłady z użyciem API.
Link: <https://platform.openai.com/docs/guides/completion>
9. **GitHub – Awesome Chatbot Projects**
Lista otwartoźródłowych projektów chatbotów z przykładami fine-tuning i prompt-engineering. Doskonały punkt startowy do nauki i eksperymentów.
Link: <https://github.com/futuga/awesome-chatbots>
10. **Artykuł o Fine-Tuning GPT-2: „How to Fine-Tune GPT-2 for Specific Applications”**
Ten artykuł wyjaśnia krok po kroku, jak dostroić GPT-2 do własnych danych, aby stworzyć niestandardowego czatbota.
Link: <https://towardsdatascience.com/how-to-fine-tune-gpt-2-12420adcf63f>
11. **LangChain – Framework dla zaawansowanego prompt-engineering i pamięci czatbota**
LangChain umożliwia budowanie chatbotów z „pamięcią” konwersacji i dynamicznymi promptami. Nadaje się do eksperymentów z narracyjnymi czatbotami.
Link: <https://www.langchain.com/>
12. **FastAPI z OpenAI GPT – Budowanie własnego API do czatbota**
Przewodnik, jak stworzyć backend do czatbota wykorzystującego OpenAI GPT z pomocą frameworka FastAPI.
Link: <https://towardsdatascience.com/building-a-chatbot-with-openai-gpt-and-fastapi-d8db5b82a0f8>
13. **Fine-Tuning GPT-3 przy użyciu OpenAI CLI**
OpenAI umożliwia fine-tuning GPT-3 z własnym zestawem danych przez interfejs wiersza poleceń. Przewodnik pokazuje, jak przygotować dane i przeprowadzić trening.
Link: <https://platform.openai.com/docs/guides/fine-tuning>
14. **Real-Time Chatbot z Flask i GPT-3**
Tutorial pokazuje, jak stworzyć prosty interaktywny chatbot za pomocą GPT-3, z integracją w czasie rzeczywistym na stronie internetowej.
Link: <https://realpython.com/flask-by-example-part-1-project-setup/>

Techniki Kontroli Odpowiedzi LLM

1. **Fine-tuning (dostrojanie modelu)**
Polega na trenowaniu modelu na specyficznych danych, aby lepiej dopasować jego zachowanie do potrzeb projektu.
 - Przykłady:
 - Dostrojanie do specyficznego uniwersum gry (np. styl mowy w świecie fantasy).
 - Dodawanie osobowości NPC (np. zgodnie z modelem Wielkiej Piątki).
 - Zalety: Bardzo precyzyjne dostosowanie.
 - Wady: Drogi i czasochłonny proces.
 - Może techniki typu Low-Rank-Adaptation?
2. **Prompt Engineering (inżynieria promptów)**
Tworzenie odpowiednich podpowiedzi bez modyfikacji modelu.
 - Przykłady:
 - Definiowanie postaci NPC w podpowiedzi, np.:
„Jesteś kowalem Galenem, który jest sceptyczny wobec obcych. Reaguj zgodnie ze swoją osobowością.”
 - Dynamiczne podpowiedzi oparte na stanie gry.
 - Zalety: Łatwość i niskie koszty.
 - Wady: Może być trudniejsze w bardziej skomplikowanych projektach.
3. **Podejścia hybrydowe**
Połączenie obu technik, np. fine-tuning dla ogólnych zachowań NPC i prompt engineering do interakcji kontekstowych.

Rekomendacje: Jeśli projekt jest mały a komputery słabe, zacznij od **prompt engineering** – jest elastyczny i szybki.

- Przy większych projektach z zasobami warto rozważyć **fine-tuning**, szczególnie gdy potrzebne są specyficzne zachowania NPC. Może techniki typu Low-Rank-Adaptation?
- Korzystajcie z otwartych narzędzi i frameworków jak **Hugging Face**, aby łatwo eksperymentować.

7) ROZWIĄZYWANIE PROBLEMU OPTYMALIZACYJNEGO: ŁAMIGŁÓWKI

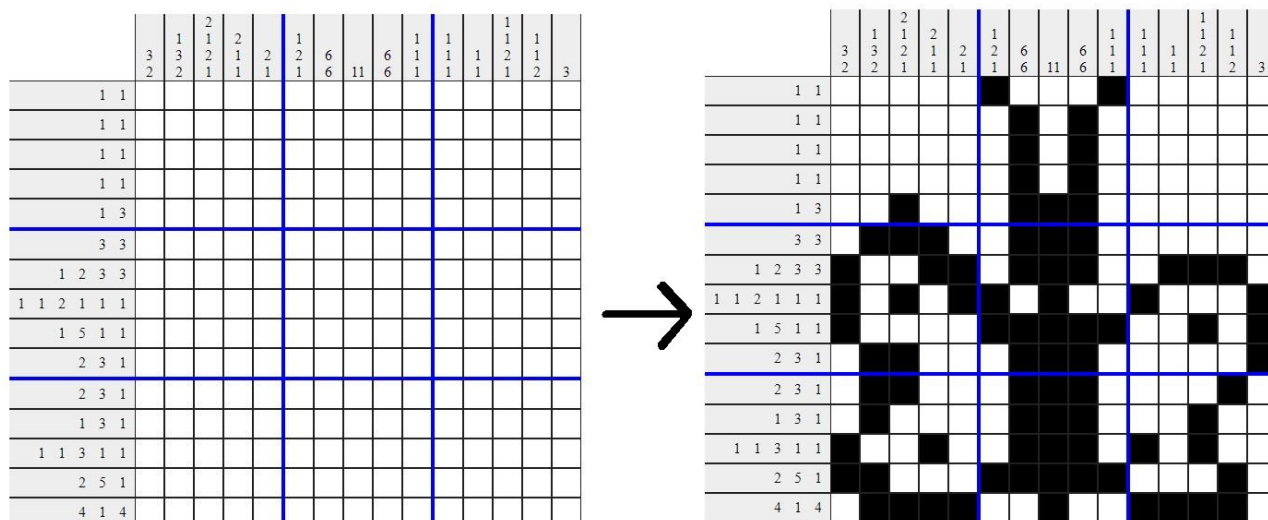
Bierzemy problem do rozwiązania np. wspomniany na wykładzie nonogram/obrazek logiczny, albo jakiś graf, a następnie szukamy rozwiązania tego problemu. Projekt polegałby na tym, że testujemy skuteczność paru algorytmów i porównujemy ich skuteczność (czy zawsze rozwiązują problem, niezależnie od jego wielkości?) oraz czas (czy wykonują się szybko?). W zależności od problemu warto porównać różne wersje algorytmu genetycznego (np. różne typy kodowania chromosomów, różne funkcje fitness), algorytm PSO lub ACO, a nawet (jeśli to możliwe) algorytmy RL. Można do tego dać jakiś bazowy algorytm typu brute force. Dobrze by było porównać 3 do 5 algorytmów dla kilku wersji problemu o różnym stopniu skomplikowania. Podsumowaniem takiego projektu mogłaby być taka tabelka z wynikami, np.:

	Przeszukiwanie grafu wszerek	Algorytm genetyczny (Fitness 1)	Algorytm genetyczny (Fitness 2)	ACO	Q-Learning
Problem 5x5	100% (1s)	100% (10s)	100% (8s)	100% (15s)	100% (20s)
Problem 10x10	100% (5s)	90% (15s)	100% (10s)	90% (20s)	100% (30s)
Problem 15x15	100% (70s)	50% (30s)	90% (15s)	60% (30s)	100% (60s)
Problem 20x20	0% (--) timeout	0% (--)	20% (60s)	10% (50s)	90% (200s)

Takie liczby powinny być średnią wielu odpaleń algorytmu (minimum 10, lepiej więcej). W przypadku gdy algorytm znalazł rozwiązanie, można zapisać jego czas działania. Można do tego zrobić też fajne wykresy z wielkością problemu na osi X (5, 10, 15, 20) i skutecznością algorytmów na osi Y (linie w różnych kolorach dla różnych algorytmów).

A jakie problemy można rozpatrzyć? Na wykładzie była mowa np. o nonogramach. Przypominamy:

Nonogram (zwany też obrazkiem logicznym) to rodzaj zagadki, w której należy zamalować niektóre kratki na czarno tak, by powstał z nich obrazek. By odgadnąć, które kratki należy zamalować, należy odszyfrować informacje liczbowe przy każdym wierszu i kolumnie krutek obrazka. Przykładowo, jeśli przy wierszu stoi "2 3 1", to znaczy, że w danym wierszu jest ciąg dwóch zamalowanych krutek, przerwa (przynajmniej jedna biała kratka), trzy zamalowane kratki, przerwa, jedna zamalowana kratka. Umieszczenie zamalowanych ciągów nie jest wskazane. Mając daną nieuzupełnioną planszę, pytamy: czy istnieje rozwiązanie dla tej zagadki (tzn. odpowiednie zamalowanie pól bez żadnych sprzeczności)? Poniżej przedstawiono instancję problemu (po lewej) i jej rozwiązanie (po prawej). Rozważ, jak znajdować rozwiązanie za pomocą algorytmu genetycznego, PSO lub innych algorytmów.



Podpowiedzi do rozwiązania:

Zanim przeczytasz odpowiedzi, zastanów się jakbyś to rozwiązał(a) samodzielnie!

Prosty pomysł na algorytm genetyczny: chromosomy mają tyle bitów, ile pól ma plansza. 1 = pole zamalowane, 0 = niezamalowane. Dla obrazka powyżej: 225 bitów.

Funkcja fitness w najprostszej postaci: policz ile reguł na brzegu planszy jest spełnionych przez rozwiązanie z chromosomu. Funkcja ta jednak jest dość prymitywna i słabo będzie naprowadzała na rozwiązania. Warto rozpatrzyć różny stopień spełnienia reguły np.

- o czy w danym rzędzie (kolumnie) jest odpowiednia liczba kratek
- o czy w danym rzędzie (kolumnie) jest odpowiednia liczba bloków krater
- o jak bardzo bloki różnią się od tych zadanych w regule

Mile widziany jest „tuning” takiej funkcji i jej odpowiednie dopasowanie. Dla powyższego pomysłu można też uruchomić algorytm BinaryPSO i zobaczyć czy działa szybciej niż algorytm genetyczny. Inny pomysł na chromosomy warty przetestowania: bloki z reguł są ustalone i wkładamy je na planszę w całości. Chromosom koduje w jakie miejsca wkładać bloki zamalowanych krater. Chromosom ma zatem m liczb naturalnych, gdzie m to liczba bloków z wszystkich reguł. To podejście ma większą szansę na zadziałanie. Nie wiadomo czy dla tego podejścia znajdzie się też jakaś wersja PSO (może Integer PSO?). Inna wersja tego problemu: Nonogramy kolorowe. Można rozpatrzyć zamiast czarnobiałych.

Inna łamigłówka: **Fill-a-pix** (zwane też mozaikami, Nurie, Nampre) to rodzaj łamigłówki podobnej do nonogramów (i trochę do sapera). Mamy planszę, na której musimy zamalować kratki na czarno, tak aby utworzyły obrazek. Niektóre kratki mają wewnątrz liczbę, która informuje, ile krater na czarno powinno być w sąsiedztwie. Maksimum sąsiadów to 9: 8 dookoła kratki i 1 to sama kratka, minimum to 0. Niektóre pola mogą być puste – nie informują o liczbie czarnych krater w sąsiedztwie, mimo że mogą je mieć.

Po prawej znajduje się przykład nierozwiązany, a pod spodem rozwiązanie: obrazek z lokomotywą.

Czy da się takie mozaiki rozwiązać algorytmami genetycznymi, PSO lub innymi algorytmami?

Zanim przeczytasz podpowiedzi, pomyśl jakbyś to rozwiązał(a).

Podpowiedź: Oczywiście można zastosować taktykę z tematu 2 o nonogramach. Chromosom to zamalowanie obrazka 1 = pole zamalowane, 0 = niezamalowane. U nas to 225 bitów. Fitness sprawdza ile reguł liczbowych jest spełnionych. Bardziej zaawansowana wersja fitness może też sprawdzać jak bardzo niespełnione są reguły (czy dużo krater brakuje, albo czy jest ich dużo za dużo, a może różnice są niewielkie?). Dla powyższego pomysłu można spróbować też uruchomić Binary PSO. Nie wiadomo czy istnieje inne, lepsze kodowanie chromosomów lub funkcje fitness. Jeśli wpadniesz na coś, koniecznie pokażesz ze swoją strategią.

2	3	3	3	4	5	5	4			0
		3	4	4						
1	2	1	2	2	3	2	2	2	4	3
	3	3	3	0					4	
			3				5	7		
5		7	6			2				
	5	7	5	5	3			5	4	1
5		7	7					7		
	7		8	9		9		9	6	
5	7	8	7	9				8	7	
4		6			6	6			7	5
	6		5	6						
	8	7		7	5	5	8	8		4
	8	7				4	8	5		2
			3	5						

2	3	3	3	4	5	5	4			0
		3	4	4						
1	2	1	2	2	3	2	2	2	4	3
	3	3	3	0					4	
			3				5	7		
5		7	6			2				
	5	7	5	5	3			5	4	1
5		7	7					7		
	7		8	9		9		9	6	
5	7	8	7	9				8	7	
4		6			6	6			7	5
	6		5	6						
	8	7		7	5	5	8	8		4
	8	7				4	8	5		2
			3	5						

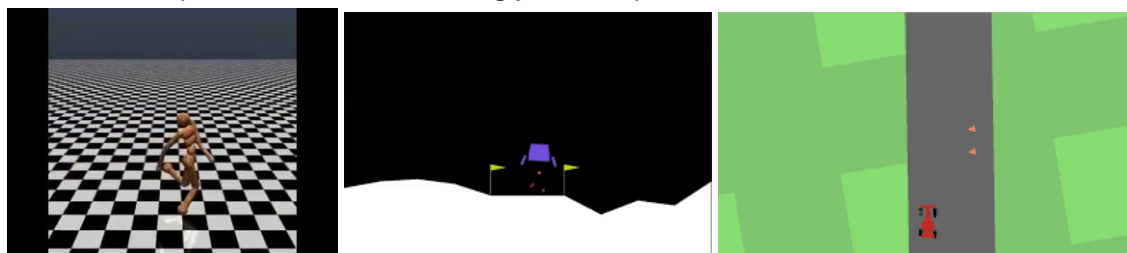
Jeśli proponowane tutaj łamigłówki Ci nie pasują, możesz poszukać innych ciekawych np. tutaj:

[https://en.wikipedia.org/wiki/Nikoli_\(publisher\)](https://en.wikipedia.org/wiki/Nikoli_(publisher))

8) STEROWANIE AGENTEM W WIRTUALNYM (LUB PRAWDZIWYM) ŚRODOWISKU

Celem projektu jest opracowanie algorytmu, który będzie dobrze sterował jakimś obiektem w danym środowisku (wirtualnym lub prawdziwym). Ponieważ nie mamy do dyspozycji robotów, dronów i innych hardwarowych kreatur, to musimy ograniczyć się do środowiska wirtualnego. Przykładem jest tu pokazana na wykładzie gra w czołgi, jak i inne gry. Można zresztą grę napisać samodzielnie i wytrenować do niej różne algorytmy. Są jednak środowiska oferujące gotowe gry lub wirtualne roboty. Przykładem jest Gym / Gymnasium

(<https://gymnasium.farama.org/index.html>) oferujące przeróżne problemy do rozwiązania: sterowanie samochodem, poruszanie robotem, różne gry arcade itp.



Gymnasium ma tę fajną cechę, że może odpalać grę wielokrotnie, trenując przy tym algorytmy. Można też wyłączyć interfejs graficzny, co przyspieszy proces uczenia.

W projekcie należałoby przetestować kilka algorytmów i porównać ich skuteczność, tj. sprawdzić jak dobrze sterują obiektem. Jeśli jest możliwość wystawienia oceny takim algorytmom (np. w ilu przypadkach doszły do celu gry, albo jak wysoko podniósł się wstający robot, albo jak daleko zaszliśmy w space shooterze) to warto zestawić te oceny w tabelce tak jak powyżej w pomyśle (1).

Jakie algorytmy warto przetestować:

- **Swój własny skrypt.** Można zapomnieć o AI i napisać prosty program do sterowania obiektem. Czasami proste rozwiązania są najlepsze. Oczywiście pytanie jest takie: czy nasz skrypt będzie na tyle elastyczny, że zadziała w każdej sytuacji i odnajdzie się w zmieniającym się środowisku?
- **Algorytmy RL.** Uczenie przez wzmacnianie, i nagradzanie agenta za wykonywanie dobrych akcji jest jedną z najpotężniejszych technik do stosowania w tym typie problemu. W zależności od typu gry i danych warto rozważyć Q-Learning lub Deep-Q-Network.
- **Kontroler rozmyty.** To wyzwanie innego gatunku: zamiast uczyć obiekt, opracowujemy rozmyty model, który ma szansę dobrze sterować agentem. Jeśli fuzzy controler umie sterować klimatyzacją czy parkować samochód, to może też będzie umiał wylądować sondą na księżycu (Lunar Lander). Taki rozmyty kontroler trzeba dobrze przemyśleć i pewnie wielokrotnie poprawiać. Warto dodać go jako technikę, zwłaszcza do problemów w których liczba zmiennych nie jest zbyt duża. Do sterowania kilkunastoma przegubami robota pewnie już będzie ta metoda za trudna do implementacji.
- **Algorytm genetyczny lub inteligencja roju.** Sterowanie obiektem można potraktować jako problem optymalizacyjny: szukamy jak najlepszego zestawu akcji, który gwarantuje dobre wykonanie akcji (np. wstanie przewróconego robota, wylądowanie sondy). Warto rozważyć, którąś z tych dwóch bio-inspirowanych metod metaheurystycznych.
- **Neuroewolucja (NEAT).** Jest technika, która wykracza poza materiał przedmiotu ale też może być bardzo ciekawa. Łączymy algorytmy genetyczne z sieciami neuronowymi w tzw. algorytmie NEAT. Algorytm genetyczny tworzy populację sieci neuronowych o różnych topologiach, a następnie ocenia ich fitness poprzez używanie ich w rozwiązywaniu problemu (np. graniu w grę).

Wybierając grę/wirtualne środowisko z Gymnasium, zwróć uwagę na to jakimi danymi opisane jest środowisko, oraz jakie akcje może wykonywać obiekt. Dane dyskretne/skończone z reguły są łatwiejsze niż zmiennoprzecinkowe.

9) ANALIZA TEKSTU Z MEDIÓW SPOŁECZNOŚCIOWYCH

Ciekawym projektem jest analiza wypowiedzi użytkowników mediów społecznościowych (Twitter/X, Facebook, Reddit, Telegram). W projekcie trzeba by zebrać dużą bazę danych postów (najlepiej powyżej 10 tysięcy). Jest to trudne, bo API wielu serwisów się zmienia lub blokuje narzędzia scrapujące. Rok temu paczka snsrape świetnie pobierała tweety, ale od paru miesięcy nie jest to możliwe. Głównym wyzwaniem tego projektu jest więc zgromadzenie dobrej i dużej bazy danych. Narzędzia do web-scrapingu (np. Selenium, BeautifulSoup) mogą się okazać dobre. Istnieją też gotowe bazy danych tweedów (np. na Kaggle) ale są one już dość mocno wyksploatowane.

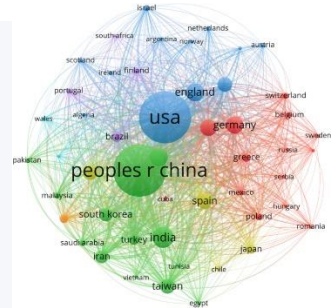
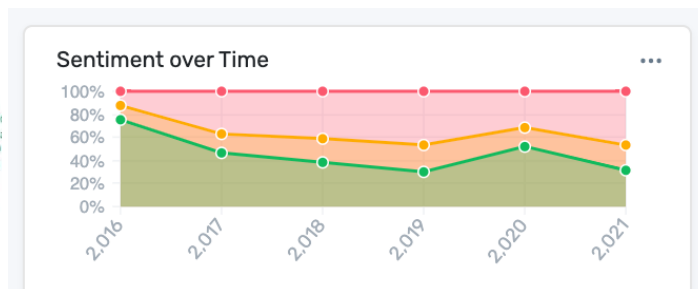
Tematyka bazy danych może być naprawdę różnorodna i obejmować różne dyscypliny życia. Przykłady pytań.

- **Revolucja AI i emocje.** Jak zmienia się nastawienie ludzi do sztucznej inteligencji na przestrzeni ostatnich miesięcy? #chatgpt #ai #genai
- **Sondaże wyborcze.** Która partia jest bardziej lubiana przed nadchodzącymi wyborami prezydenckimi 2025 Czy natężenie opinii pokrywa się z sondażami wyborczymi? #Nawrocki #Trzaskowski ?
- **Musk vs Bitcoin, Musk vs Tesla.** Czy opinie nt Elona Muska wpływają na akcje Tesli lub cenę Bitcoina? Sprawdź czy istnieje korelacja w czasie. #Musk #ElonMusk #Tesla #Bitcoin
- **Witcher? Rings of Power? Game of Thrones?** Wybierz popularny serial, najlepiej budzący różne emocje i sprawdź jakie emocje wywołuje wśród użytkowników mediów społecznościowych. Można pomierzyć emocje przed i po konkretnym sezonie, lub konkretnym odcinku i sprawdzić jak zmienia się nastawienie.

Jak widać tematy mogą być polityczne, z showbiznesu, rozrywki czy finansów. Wybierz taki, który Cię szczególnie interesuje. Opinie najlepiej zebrać z różnych okresów w czasie. Musisz przemyśleć jak je rozbić? Z różnych dni/tygodni/miesięcy? A może wystarczy rozbić na dwa okresu: przed i po jakimś wydarzeniu.

Zebrane posty można analizować pod różnymi kątami:

- Jakie słowa w nich występują? Bag of words, Word cloud, Word frequency.
- Jaki wydźwięk mają? Vader/Text2Emotion itp. Czy wydźwięk się zmienia w czasie? (Wykres liniowy).
- Analiza tematyki opinii i klasteryzacja.



10) PREDYKCJA LUB GENERACJA DLA SZEREGÓW CZASOWYCH (TEKST, MUZYKA, MOWA, GIEŁDA, ITP.)

Jak tytuł wskazuje, chcemy wybrać jakiś typ danych:

- Tekst (z jakiego źródła? Książki? Źródła internetowe?)
- Muzyka (MIDI? MP3? Jaki format? Jakie źródło?)
- Dźwięk (Mowa? Odgłosy zwierząt?)
- Dane finansowe (Ceny akcji?)

Następnie na podstawie tych danych możemy wyuczyć jakiś model przewidywać lub generować kolejne słowa/dźwięki/nuty/itp., albo klasyfikować dany obiekt (pisarz dla książki, genre dla kawałka muzycznego, gatunek ptaka dla odgłosu, ryzyko inwestycyjne dla akcji giełdy, itp.).

W tym projekcie warto przetestować poznane modele deep learning, czyli:

- Sieci rekurencyjne (zwykłe).
- Sieci LSTM.
- Transformery.

Sz szczególnie interesujące (zwłaszcza w kontekście tekstu) może być wykorzystanie gotowych modeli (np. pobranych z huggingface) i dotrenowanie ich na naszej bazie danych. Taki fine-tuning może być jednak obciążający czasowo, ale są różne techniki, które przyspieszają trenowanie, np. LoRA (Low Rank Adaptation). W projekcie warto uwzględnić jeden taki eksperyment z fine-tuningiem.

W tym projekcie kuszące są różne zastosowania aplikacyjne. Przykład: trenuję chatbota, którego umieszczam w grze RPG jako postać NPC. Gracz może rozmawiać z taką postacią na różne tematy, ale ważne żeby chatbot

zachowywał się jakby był faktycznie postacią w RPGu. Chatbot na stronę UG, który pomagałby użytkownikom strony znaleźć przydatne informacje też byłby ciekawym pomysłem.

Jeśli chodzi o modele tekstowe to dość popularne są:

- Llama <https://huggingface.co/meta-llama/Meta-Llama-3-8B>
- Mistral https://huggingface.co/docs/transformers/model_doc/mistral
- Polski model: Bielik <https://huggingface.co/speakleash/Bielik-7B-Instruct-v0.1>

11) PRZESZUKIWANIE STREAMÓW / DŁUGICH VIDEO: WYSZUKIWARKA MOMENTÓW + HIGHLIGHTY + AUTO-PRZEWIJANIE

1) Wprowadzenie, motywacja, inspiracje

Długie video (VOD) i streamy mają jeden wspólny problem: **wartościowe momenty są schowane w godzinach materiału**. Przez to:

- twórca traci czas na ręczne wycinanie “highlightów”,
- widz traci czas na przewijanie,
- w monitoringu (CCTV) człowiek nie jest w stanie sensownie “oglądać wszystkiego”, a często zależy nam na **jednym konkretnym zdarzeniu**.

Ten projekt to prototyp “wyszukiwarki video”, która potrafi:

1. **wyciągnąć highlighty** (najciekawsze fragmenty),
2. **pozwolić wyszukać momenty po zapytaniu** (np. “osoba zostawia walizkę”, “auto parkuje na zakazie”, “pojawili się pisklęta w gnieździe”),
3. **automatycznie przewijać nudne fragmenty** (np. statyczne ujęcia bez akcji / bez mowy).

To bardzo dobrze pasuje do formuły “aplikacja/serwis + element badawczy”: robisz działające demo, ale w raporcie pokazujesz, *co testowałeś, co działało lepiej i dlaczego* (porównania metod/ustawień/metryk).

Use-case’y (żeby projekt miał “pazur”):

- **Streamy / YouTube / wykłady**: “znajdź moment, gdy pokazano slajd z wzorem”, “kiedy padło hasło ‘overfitting’”, “pokaż najlepsze 10 akcji”.
- **CCTV lotnisko**: “czy ktoś zostawił walizkę i odszedł?” (abandoned baggage).
- **CCTV parking**: “czy ktoś stanął na nielegalnym miejscu i ile stał?” (parked vehicle).
- **Kamera przyrodnicza**: “czy w gnieździe pojawiły się pisklęta / karmienie / przylot dorosłego ptaka”.

Inspiracje (linki na zajawkę):

- Demo “semantic video search” (szukanie scen w video jak w Google): [YouTube](#)
- Marqo: przykład zrobienia wyszukiwarki klipów w YouTube z timestampami (blog + demo repo): [marqo.ai+1](#)
- PySceneDetect – gotowe narzędzie do dzielenia video na sceny (przydaje się do rozdziałów i “nudnych” fragmentów): [scenedetect.com+1](#)

2) Technika: biblioteki, technologie, sprzęt, dane, trenowanie

Ogólny pomysł (pipeline)

1. **Video/stream** dzielisz na krótkie kawałki (np. co 1–2 sekundy) lub na sceny.
2. Z każdego kawałka wyciągasz “opis”:
 - co widać (obiekty, osoby, pojazdy),
 - co się dzieje (ruch, interakcje),
 - co słychać (mowa / nagłe dźwięki).
3. Te opisy indeksujesz tak, żeby potem:
 - znaleźć fragmenty pasujące do zapytania,
 - ocenić “ciekawość” fragmentu i złożyć highlighty,
 - wykryć długie odcinki “nic się nie dzieje” i je pominąć.

Biblioteki (Python) – propozycja zestawu

- **OpenCV** – czytanie video, klatki, zapis klipów, proste filtry.
- **YOLO / detekcja obiektów** (np. Ultralytics YOLO) – ludzie, walizki, auta, itp.
- **Tracker** (np. ByteTrack/DeepSORT) – żeby wiedzieć, że to “ta sama osoba” przez wiele sekund.
- **PySceneDetect** – wykrywanie zmian scen, rozdziały i cięcie video. [scenedetect.com+1](#)
- **Whisper (ASR)** – transkrypcja mowy (dla wykładów/streamów).

- **CLIP / embeddingi obrazów + FAISS/Milvus** – jeśli chcesz wyszukiwać “po znaczeniu” (zapytanie tekstowe → podobne klatki/fragmenty). (To da się zrobić w wersji “light” bez trenowania od zera.)

Ważne: da się zrobić MVP nawet bez ciężkich modeli “akcji” – samo połączenie detekcji+trackingu + prostych reguł + transkrypcji już daje użyteczne wyszukiwanie.

Sprzęt

- Do prototypu: zwykły laptop wystarczy (na krótkich filmach).
- Do płynnego działania na długich nagraniach: pomaga GPU (ale nie jest konieczne, jeśli ograniczysz FPS lub przetwarzasz co N klatek).

Dane: skąd wziąć?

Masz 3 opcje (można je mieszać):

A) Dane własne (najłatwiejsze organizacyjnie)

- Ściągasz kilka długich filmów (np. streamy, wykłady) i budujesz indeks.
- Do CCTV/przyrody: możesz użyć publicznych streamów lub gotowych nagrań (albo symulować scenki: ktoś zostawia plecak, auto parkuje).

B) Klasyczne datasety pod “walizka/parking” (CCTV)

- **AVSS 2007** – zawiera zadania “abandoned baggage” i “parked vehicle”. eecs.qmul.ac.uk+1
- **PETS 2006** – często używany do scen z pozostawionym bagażem. media-lab.ccny.cuny.edu+1

To są świetne “case’y” do pokazania, że system działa nie tylko na YouTube, ale też na monitoringu.

C) Dane do highlightów

- Najczęściej highlighty są subiektywne, więc:
 - robisz prostą “prawdę referencyjną” sam: oznaczasz 30–60 minut materiału i zaznaczasz ciekawe momenty,
 - albo robisz mini-badanie użytkowników: 2–3 osoby wskazują highlighty i porównujesz zgodność.

Trenowanie (jak to ugryźć w projekcie)

- **MVP bez trenowania:**
 - “ciekawość” = nagły ruch, dużo zmian w kadrze, wzrost głośności, przyrost mowy, dużo obiektów.
 - “nudne” = statyczny kadr + brak mowy + brak nowych obiektów przez X sekund.
 - “zostawiona walizka” (wersja prosta) = walizka pojawia się, osoba odchodzi, walizka zostaje nieruchoma przez X czasu.
- **Wersja ambitniejsza (trochę uczenia):**
 - uczysz klasyfikator “ciekaw / nieciekaw” na cechach (ruch, audio, liczba obiektów, tempo zmian scen),
 - albo uczysz rozpoznawanie zdarzeń (np. “pozostawienie obiektu”, “nielegalny postój”) na krótkich klipach.

3) Końcowy produkt: minimalne funkcje + dodatki

MVP (żeby dało się dobrze pokazać na obronie)

1. **Wrzucasz plik wideo** (albo link do źródła) i system robi indeks.
2. **Wyszukiwarka:** wpisujesz zapytanie (np. “osoba z walizką”, “samochód parkuje”) → dostajesz listę fragmentów z **timestampami** + miniaturką.
3. **Highlight generator:** “stwórz mi top-10 klipów po 10–20 s”.
4. **Auto-przewijanie:** suwak “agresywność pomijania nudy” + lista odcinków, które uznano za nudne.

Dodatki (robią “wow” i są dobre do raportu porównawczego)

- **Tryb CCTV:** gotowe scenariusze:
 - “pozostawiony bagaż” (alarm, gdy obiekt jest bez właściciela),
 - “nielegalny postój” (ile minut stoi, czy pojazd zasłania wjazd).
- **Tryb “rozdziały”:** automatyczne chaptery (sceny + transkrypcja) – przydatne do wykładów.
- **Wyjaśnienia:** dlaczego system uznał moment za highlight (np. “nagły wzrost ruchu + pojawił się nowy obiekt + głośny dźwięk”).
- **Porównanie metod** w raporcie: PySceneDetect vs stałe okienka czasowe, reguły vs lekki model uczony, różne progi, wpływ FPS itp. (t